Learning a Catan Agent with Maskable PPO and Reward Shaping

Arrush Shah Suraj Kidiyoor
Syslab P3
Dr. Yilmaz

***Abstract*—Settlers of Catan is a challenging benchmark for game-playing agents because it combines stochastic dice rolls, imperfect long-term planning, spatial placement, trading, and many legal-action constraints. This paper describes a reinforcement learning agent built on top of Catanatron using Stable-Baselines3 and `sb3-contrib`'s MaskablePPO. The implementation represents each game state as a 614-dimensional feature vector, masks illegal actions from a 290-action discrete action space, and trains a policy/value neural network with separate policy and value heads. Several reward functions are compared, beginning with a sparse terminal reward and moving toward shaped rewards based on victory point progress, settlement construction, and city construction. The final presentation results show that reward design had a major effect on performance: the improved reward reached a 66% win rate in one-hour tests against three random bots, while the basic reward stayed near the 25% four-player random baseline. Longer training and curriculum testing improved performance against Random, WeightedRandom, and Monte Carlo Tree Search opponents, but the model still struggled against the stronger ValueFunction bot. These results suggest that reward shaping and valid-action masking are necessary for this project, while stronger opponent modeling, better reward diagnostics, and improved board representations remain important future work.**

***Index Terms*—reinforcement learning, Catan, MaskablePPO, reward shaping, game AI, Catanatron**

I. Introduction

Catan is a useful game for reinforcement learning (RL) because the agent must make decisions under uncertainty over a long horizon. A player must choose initial settlements, expand roads, allocate resources, decide when to buy development cards, trade with opponents or the bank, and respond to the robber. Unlike deterministic board games, Catan includes dice rolls, hidden or partially hidden strategic information, and changing board conditions. A simple fixed algorithm is difficult because the best move depends on resources, port access, expansion space, opponent pressure, and future dice probabilities.

The goal of this project was to build a Catan bot, called CaRL, that learns from simulated play rather than relying only on fixed rules. The project used Catanatron as the simulator and Gymnasium-compatible environment, Stable-Baselines3 for the RL framework, and MaskablePPO for action masking. The agent was trained and evaluated using several reward functions. The best reward gave terminal win/loss rewards and mid-game rewards for victory point, settlement, and city progress.

This paper makes four contributions. First, it documents the current Catanatron-based MaskablePPO pipeline used in the project. Second, it explains how the 614-feature state and 290-action output are used by the policy and value networks. Third, it updates the experimental section using the final presentation results and figures. Fourth, it identifies implementation limitations that explain why the model can beat simpler bots but still struggles against stronger strategy-based opponents.

II. Background

A. Catan as an RL Problem

Catan is a multi-agent, stochastic, turn-based strategy game. The objective is to reach ten victory points (VP). Points are mainly earned by building settlements, upgrading cities, holding victory point development cards, and obtaining bonuses such as Longest Road and Largest Army. A good Catan player must balance short-term resource collection with long-term board position.

From an RL perspective, each decision can be written as a Markov decision process approximation:

$$s_t \to a_t \to r_t \to s_{t+1}, \tag{1}$$

where s_t is the encoded board and player state, a_t is the selected action, and r_t is the reward returned by the environment. The true game is multi-agent and partially observable, so this formulation is an approximation. In the current project, the agent trains from a vectorized environment and receives a dense or sparse reward depending on the reward function.

B. Why Action Masking Matters

The Catan action space is highly constrained. At any state, only a small subset of all possible discrete actions is legal. For example, a player cannot build a road on an occupied edge or buy a development card without the required resources. Training PPO without handling invalid actions wastes probability mass on illegal moves and can produce noisy gradients. MaskablePPO solves this by setting illegal actions to zero probability before sampling or selecting an action. If $A(s)$ is the legal action set, the masked policy can be written as

$$\pi_\theta(a_i \mid s) = \frac{\exp(z_i)\mathbb{1}[i \in A(s)]}{\sum_j \exp(z_j)\mathbb{1}[j \in A(s)]}, \tag{2}$$

where z_i is the policy logit for action i.

C. PPO and Policy/Value Learning

Proximal Policy Optimization (PPO) updates a stochastic policy while limiting how far each update moves from the previous policy. The policy head predicts an action distribution and the value head predicts the expected return from the current state. In this project, both heads use the same flattened feature input but have different final layers: one returns a score for each action and the other returns a scalar value estimate.

III. Related Work

Three common approaches to Catan AI are rule-based bots, search-based methods, and learning-based methods. Rule-based bots, such as JSettlers2, can perform well when their heuristics encode strong strategy, but they do not automatically adapt to new opponents or discover new policies. Search-based methods such as alpha-beta pruning, A*, and Monte Carlo Tree Search are difficult because Catan has stochastic dice rolls, hidden information, negotiation, trading, and many state-dependent legal actions. Supervised learning is limited by the lack of large public expert datasets for Catan games. Reinforcement learning avoids the need for labeled expert moves, but it introduces its own challenges: reward sparsity, long episodes, unstable training, and high simulation cost.

Catanatron provides the core game simulator, a command-line evaluation interface, built-in bots such as Random, WeightedRandom, ValueFunction, and Monte Carlo Tree Search, and a Gymnasium-compatible RL interface. The project in this paper builds on that environment rather than implementing Catan rules from scratch.

IV. System Design

A. Repository and Runtime Stack

The repository contains a modified Catanatron project with training scripts, saved models, checkpoints, a web Dockerfile, and a React UI. The main training script is `MYMODEL.py`; the CLI integration script is `PLAYMYMODEL.py`. The requirements include `numpy`, `torch`, `stable-baselines3`, `sb3-contrib`, and `gymnasium`. The web stack uses a Python backend and a React frontend connected through Docker and hosted through Render web services.

B. Environment

The training environment is created with the Catanatron Gymnasium environment and then wrapped with an action masker:

```
env = gym.make(
   "catanatron/Catanatron-v0",
   config={
      "reward_function": reward,
      "invalid_action_reward": -1.0,
   },
)
env = ActionMasker(env, mask_fn)
```

The reward function is passed directly into the Catanatron environment. The `ActionMasker` wrapper calls a custom mask function that queries Catanatron for the current valid actions.

C. Observation Representation

The project identifies the game state as a 614-value vector containing board information, player resources, victory points, roads, buildings, tile resources, dice locations, ports, and cards. This vector is flattened and used as the neural network input. In the CLI player, `create_sample(game, color)` and `get_feature_ordering(num_players=2)` are used to reconstruct the same feature order for model inference.

D. Action Space

The environment uses a discrete action space of size 290. Each output index corresponds to a possible Catan action under Catanatron's action mapping. During live CLI play, the player converts playable Catanatron actions into action-space indices using `to_action_space`. The model predicts an action index, and the script maps the predicted index back to the original playable action.

E. Neural Architecture

The model uses an MLP policy with two hidden layers of 64 units. The policy and value networks share the same input size but differ in the output layer:

$$\text{policy}: \quad 614 \to 64 \to 64 \to 290, \tag{3}$$

$$\text{value}: \quad 614 \to 64 \to 64 \to 1. \tag{4}$$

The 290-dimensional policy output is interpreted as move logits before masking and sampling. The scalar value output estimates the expected return from the current state. Figure 1 shows the overall training loop used in the presentation: the encoded game state passes through the policy/value model, a legal move is selected, the game checks whether that move eventually helped the agent win, and the model weights are updated before the next training cycle.

F. Code Repositories and Reader Guides

The project code is split across two public GitHub repositories. The main training and evaluation code is available in the CaRL/Catanatron project repository: https://github.com/Zawiop/canatron-syslab. The playable website/demo code is available in the website repository: https://github.com/professorcool09/CatanWebsite. The Overleaf submission package also includes two companion reader guides, `README-canatron-syslab.md` and `README-CatanWebsite.md`, so that readers can quickly understand which files contain the training loop, saved models, web backend, templates, and model-loading adapter.

G. Website Interface

A major project component was the playable website interface. The frontend allows users to interact with the trained model visually instead of only running command-line tests. The backend connects the user interface to the Catanatron environment and trained policy. The final presentation notes that the website is hosted online through Render web services, but the current hosted version can only support one active game at a time. This interface is important because it turns the

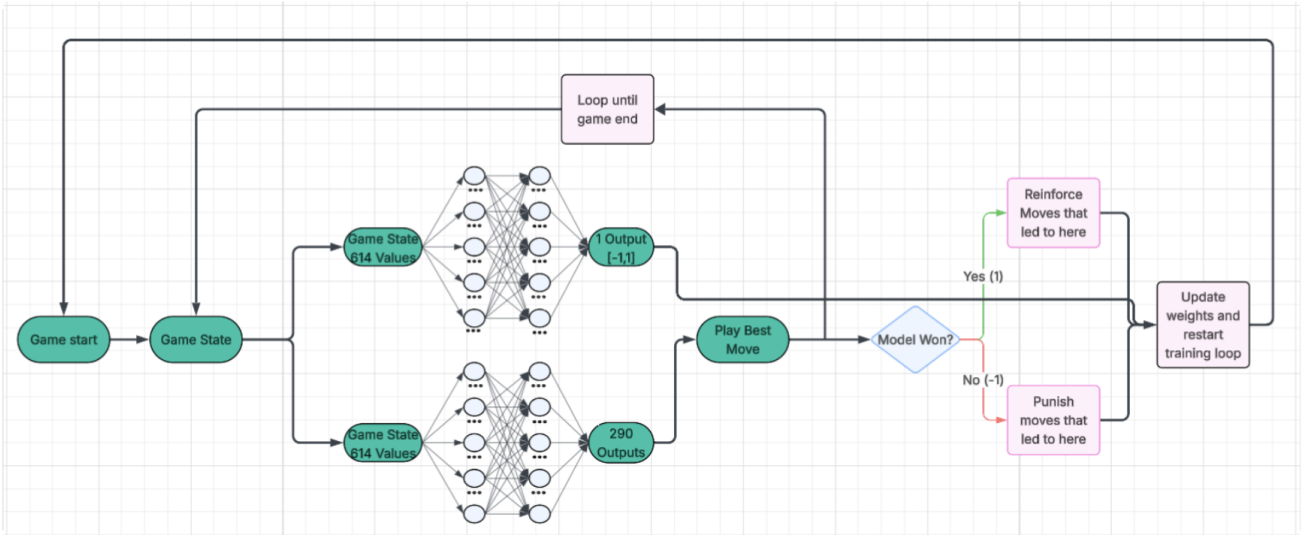

Fig. 1: Updated presentation diagram of the CaRL training pipeline. A 614-value game-state vector is processed by the policy and value networks, masked to legal moves, evaluated through game outcomes, and then used to update model weights over repeated games.

model from a terminal-only experiment into a demonstration that other users can test.

V. Implementation

A. Valid-Action Masking

The mask function creates a Boolean vector of length `env.action_space.n`. It marks all valid Catanatron action indices as true and all other actions as false:

```
def mask_fn(env):
    n = env.action_space.n
    valid = np.array(env.unwrapped.get_valid_actions(),
                     dtype=np.int64).flatten()
    mask = np.zeros(n, dtype=np.bool_)
    valid = valid[(valid >= 0) & (valid < n)]
    if valid.size == 0:
        raise RuntimeError("Zero valid actions")
    mask[valid] = True
    return mask
```

B. Training Configuration

Training uses parallel subprocess environments and normalizes observations and rewards:

```
N_ENVS = 8
venv = SubprocVecEnv([make_env for _ in range(N_ENVS)])
venv = VecNormalize(venv, norm_obs=True,
                    norm_reward=True, clip_obs=10.0)
model = MaskablePPO(
    "MlpPolicy", venv,
    learning_rate=1e-4,
    ent_coef=0.005,
    n_steps=1024,
    batch_size=512,
    gamma=0.99,
)
```

The script saves checkpoints during training and saves the final model as `MYMODEL_final<TIMESTEPS>`. The training length is controlled by an editable `hours` variable, where the code converts hours to steps as `5,000,000 * hours`.

C. Inference Player

The trained model is exposed as a Catanatron CLI player named `PPO`. At decision time, the inference script rebuilds the 614-feature observation, constructs an action mask from `playable_actions`, and calls

```
action_idx, _ = self.model.predict(
    obs,
    action_masks=mask,
    deterministic=True,
)
```

The model is therefore stochastic during training but deterministic during evaluation. If the predicted action cannot be mapped back to a legal action, the script falls back to a playable action.

VI. Reward Design

A. Sparse Terminal Reward

The initial reward function returned zero during the game and only rewarded the terminal result:

$$r_{basic} = \begin{cases} +1, & \text{if the agent wins,} \\ -1, & \text{if the agent loses,} \\ 0, & \text{otherwise.} \end{cases} \tag{5}$$

This reward is simple and aligned with the real objective, but it is very sparse. Since a Catan game can last hundreds of turns, the agent receives little feedback about which earlier moves caused the final win or loss.

B. Long-Term Reward Attempt

A more complex reward tracked actual victory points, public victory points, Longest Road, and Largest Army. It also applied a small step penalty to encourage faster wins:

$$\begin{aligned} r_{long} = &-0.01 + 1.5\Delta VP_{actual} + 0.3\Delta VP_{public} \\ &+ 0.75\Delta LR + 0.75\Delta LA + r_{terminal}. \end{aligned} \tag{6}$$

The intended terminal reward was large, approximately $+20$ for a win and -20 for a loss. This version was conceptually closer to Catan strategy, but it performed poorly in the reported experiments. The likely problem is that too many moving reward terms can make credit assignment noisy, especially if the reward state is not reset perfectly between games.

C. Intermediate Victory Point Reward

A third reward reduced complexity by tracking only victory point changes plus terminal reward:

$$r_{vp} = 0.2\Delta VP + \begin{cases} +10, & \text{win}, \\ -10, & \text{loss}, \\ 0, & \text{non-terminal}. \end{cases} \tag{7}$$

This improved on the long-term reward attempt, but it still did not directly reward board development that often precedes future victory points.

D. Improved Reward

The strongest reward used a shaped reward based on victory points, settlements, and cities:

$$\begin{aligned} r_{improved} =& 2.5\Delta VP + 1.5\Delta S + 2.5\Delta C \\ &+ \begin{cases} +10, & \text{win}, \\ -10, & \text{loss}, \\ 0, & \text{non-terminal}, \end{cases} \end{aligned} \tag{8}$$

where S is the number of settlements and C is the number of cities controlled by the agent. This reward stays close to the real scoring system while giving the agent earlier feedback for progress that usually leads to future points.

VII. Experiments

A. Evaluation Setup

The project reports several evaluations. The first evaluations tested models against random players after 5 million and 20 million training steps. Later evaluations compared reward functions after one hour of training. The final presentation added curriculum-style training and testing against stronger opponents: first against three random bots, then with WeightedRandom bots, then with ValueFunction bots. The final testing slides compare the model against Random, WeightedRandom, Monte Carlo Tree Search, and ValueFunction opponents at several training checkpoints.

Because the experiments were run during development, they should be interpreted as exploratory rather than definitive. Some tests used 100 games, others used 500 or 1000 games, and the strongest-bot tests used smaller samples. Random seeds, confidence intervals, and repeated independent training runs were not reported, so the results show project direction rather than final statistical proof.

TABLE I: Reported 100-game results against random opponents.

Setting	Red	Blue	Orange	RL/PPO
5M steps	27%	26%	22%	25%
20M steps	10%	11%	12%	67%

TABLE II: Reported reward comparison after one hour of training.

Reward function	Win rate
Basic terminal reward	25%
Long-term shaped reward	7%
Third reward	36%
Improved reward	66%

B. Training Curriculum

The final presentation describes a harder-opponent curriculum. The model was first trained against three random bots for 10 million steps, then against one WeightedRandom bot and two random bots for 6 million steps, then against three WeightedRandom bots for 2 million steps. After that, the agent was trained with ValueFunction opponents: one ValueFunction bot with two WeightedRandom bots for 2 million steps, and then two ValueFunction bots with one WeightedRandom bot for another 2 million steps. This curriculum was designed to expose the agent to stronger play without making early training too slow or too difficult.

C. Metrics

The primary metric is win rate. Secondary logged training metrics include mean episode reward, total timesteps, frames per second, approximate KL divergence, entropy loss, policy gradient loss, value loss, and explained variance. During game evaluation, the project also recorded average victory points, settlements, cities, roads, army size, and development cards.

VIII. Results

A. Sparse Reward Baseline

The sparse baseline initially behaved close to random. After about 81 seconds and 100,352 timesteps, the reported training reward showed no useful improvement. After 5 million steps, the RL player won 25 out of 100 games, which is approximately the random expectation in a four-player game. After 20 million steps, one reported model won 67 out of 100 games against random opponents, showing that longer training could produce improvement even before the final reward function was selected.

B. Reward Function Comparison

The reward comparison after one hour of training showed that the improved shaped reward performed best. The reported win rates were 25% for the basic reward, 7% for the long-term reward, 36% for the third reward, and 66% for the improved reward. Figure 2 visualizes this comparison from the updated presentation.

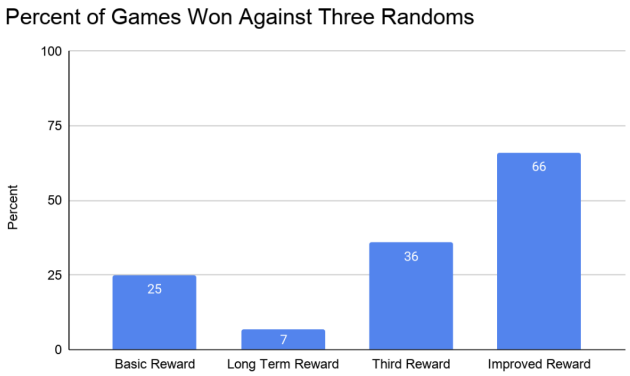

Fig. 2: Updated presentation chart comparing reward functions after one hour of training against three random bots. The improved reward was the strongest, reaching a 66% win rate.

TABLE III: Approximate final-presentation testing summary.

Opponent type	Overall result
Random	Strong performance, about mid-90% win rate
WeightedRandom	Strong performance, about low-80% win rate
Monte Carlo Tree Search	Competitive, roughly 80–90% in the slide results
ValueFunction	Weak, about 5–10% in the slide results

C. Testing Throughout Training

The updated presentation shifts the main longer-training result away from only comparing one-hour, four-hour, and eight-hour random-opponent runs. Instead, it shows testing at 10M, 16M, 18M, and 22M steps against increasingly difficult opponents. Figure 3 shows that the model performed strongly against Random and WeightedRandom bots, performed competitively against Monte Carlo Tree Search, and remained weak against ValueFunction. This result is more informative than random-only testing because it separates performance against simple stochastic opponents from performance against stronger strategy-based opponents.

D. Evaluation Against Stronger Bots

The final results show that CaRL improved beyond random play, but did not surpass the strongest Catanatron heuristic model. Against Random and WeightedRandom opponents, the agent performed far above a four-player random baseline. Against Monte Carlo Tree Search, the final presentation reports that the model won roughly 80% of games, suggesting that the neural policy can be much faster to evaluate while still choosing competitive moves. Against ValueFunction, the model won only about 5% of games, showing that the current approach still lacks the strategic depth of the strongest heuristic opponent used in testing.

IX. Discussion

A. Why Reward Shaping Helped

The sparse terminal reward makes every non-terminal decision look equally neutral. This is especially difficult in Catan because building a settlement, upgrading a city, or improving production may not directly end the game, but these actions strongly affect future win probability. The improved reward helps by giving immediate feedback for board-development progress. Settlement and city rewards also align with Catan's actual scoring and resource-production mechanics.

B. Why Stronger Opponents Remain Difficult

The updated results suggest that the model learned a useful policy, but mostly against opponents with weaker or more random move selection. ValueFunction remains difficult because it evaluates board quality more directly and makes stronger strategic choices. Training against ValueFunction is also slower because that bot must evaluate its board and choose a move on every turn; the final presentation notes that each ValueFunction opponent is about five times slower than Random or WeightedRandom during training. This makes curriculum training necessary but expensive.

C. Limitations

The current project has several limitations. First, most experiments are not repeated with multiple random seeds. Second, some strong-opponent evaluations use fewer games than the random-opponent evaluations. Third, the agent has not fully solved opponent trading or hidden-information reasoning. Fourth, the model uses a simple MLP over flattened features rather than a graph or board-aware architecture. Fifth, the hosted website currently supports only one active game at a time. These limitations matter because the model's high win rates against simpler bots do not necessarily prove human-level Catan strategy.

X. Future Work

The next version should make the training loop more reliable and improve performance against stronger opponents. The highest-priority changes are:

1) Log reward components separately so that VP, settlement, city, and terminal rewards can be debugged independently.
2) Evaluate with fixed seeds, multiple independent training runs, and confidence intervals.
3) Use MaskablePPO-specific evaluation utilities when evaluating masked policies.
4) Improve the reward function with higher-level Catan strategy, such as rewarding strong initial settlements, production diversity, port access, and expansion potential.
5) Implement fuller trading with opponents instead of mainly relying on bank trading.
6) Train longer on stronger hardware so that ValueFunction and mixed-opponent curricula become practical.
7) Consider a graph neural network or board-structured representation instead of a flat MLP.
8) Improve the website backend so more than one game can be played at the same time.

XI. Conclusion

This project demonstrates that MaskablePPO can be integrated with Catanatron to train a working Catan agent. The agent uses a 614-feature state vector, a 290-action discrete

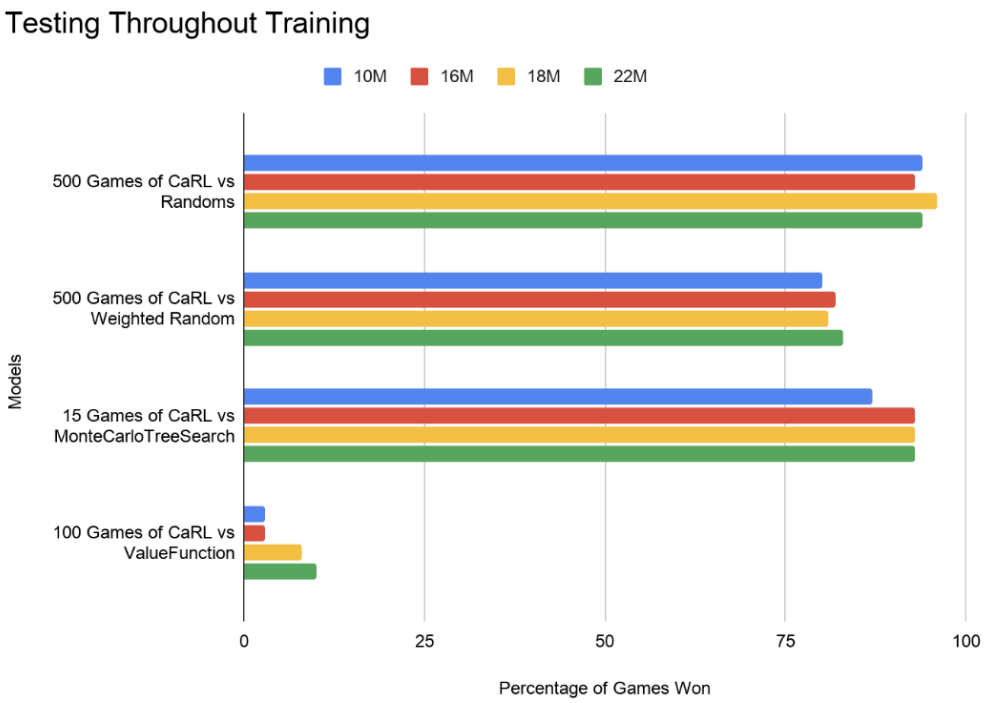

Fig. 3: Updated presentation chart for testing throughout training. The model was tested against Random, WeightedRandom, Monte Carlo Tree Search, and ValueFunction opponents at 10M, 16M, 18M, and 22M training steps.

action space, valid-action masking, vectorized environments, and a policy/value MLP. The experiments show that sparse win/loss reward is not enough for reliable learning, while reward shaping based on victory points, settlements, and cities can produce much stronger results. The final presentation results show strong performance against Random, WeightedRandom, and Monte Carlo Tree Search opponents, but weak performance against ValueFunction. The project is therefore best understood as a functional RL pipeline and a promising prototype rather than a solved Catan bot. With better reward diagnostics, stronger curricula, opponent trading, robust evaluation, and board-aware representations, the system can become a more reliable Catan learning agent.

Appendix A
Key Implementation Details

A. *RewardIteration2*

The improved reward tracks the previous victory point count, settlement count, and city count. On each step, it rewards positive changes and penalizes terminal losses:

```
reward = 0.0
reward += 2.5 * (vp - self.prev_vp)
reward += 1.5 * (settlements - self.prev_settlements)
reward += 2.5 * (cities - self.prev_cities)
if winner is not None:
   return 10.0 if winner == p0_color else -10.0
```

A production version should reset stored previous values using an explicit episode boundary rather than relying only on object state.

B. *PPO Player Registration*

The CLI player registers the trained policy under the name `PPO`, making it possible to run simulations through Catanatron's command-line interface:

```
register_cli_player("PPO", PPOPlayer)
```

This allows batch testing against Random, WeightedRandom, ValueFunction, Monte Carlo Tree Search, and other Catanatron bots.

References

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," arXiv:1707.06347, 2017.

[2] B. Collazo, "Catanatron: Settlers of Catan Bot Simulator and AI Player," GitHub and documentation. Available: https://github.com/bcollazo/catanatron and https://docs.catanatron.com/

[3] Farama Foundation, "Gymnasium Documentation." Available: https://gymnasium.farama.org/

[4] Stable-Baselines3 Contributors, "PPO - Stable-Baselines3 Documentation." Available: https://stable-baselines3.readthedocs.io/en/master/modules/ppo.html

[5] Stable-Baselines3 Contrib Contributors, "Maskable PPO - SB3 Contrib Documentation." Available: https://sb3-contrib.readthedocs.io/en/master/modules/ppo_mask.html

[6] JSettlers2 Contributors, "JSettlers2," GitHub repository. Available: https://github.com/jdmonin/JSettlers2

[7] A. Shah and S. Kidiyoor, "canatron-syslab," GitHub repository. Available: https://github.com/Zawiop/canatron-syslab

[8] A. Shah and S. Kidiyoor, "CatanWebsite," GitHub repository. Available: https://github.com/professorcool09/CatanWebsite

[9] A. Shah and S. Kidiyoor, "CaRL — Solving Catan using RL," Syslab P3 project presentation, May 2026.